

Direct Evaluation of Time-Decaying Walk Kernels for Music Recommendation: Complexity, Cold Start, and Non-Stationarity versus Embedding-Based Retrieval

July 2026 — draft

Status: paper-style reference material — there is no behavioral contract here. GitHub’s mark-down math rendering is unreliable for this document — read the typeset **PDF version** instead. This is the information-recommendation (music) framing of the argument; the companion note [graph-proximity-vs-embeddings.md](#) develops the same mathematics in a retrieval-systems framing, and [illuminate-complexity.md](#) compares the traversal against B-tree self-joins. Reference implementation: Lantern (§9).

Abstract. Two families of systems answer “what should this listener hear next?” from the same implicit-feedback stream. *Embedding-based retrieval* trains item vectors (item2vec on listening sessions, matrix factorization on play counts, audio encoders) and serves top- k inner products from an approximate-nearest-neighbor (ANN) index. *Graph-based retrieval* maintains an explicit interaction graph — tracks linked by co-listening, playlist co-occurrence, or sequential transition — and serves bounded traversals: truncated walks, Personalized PageRank (PPR), local communities. We make precise the sense in which these are two estimators of one mathematical object, a *walk kernel* $M = \sum_k c_k P^k$ over the item set. A line of theorems (Levy–Goldberg [12], NetMF [22], HOPE [21]) shows the standard embedding families are implicit or explicit rank- d factorizations $\Phi\Phi^\top \approx f(M)$ of such kernels; conversely, the dominant ANN indexes answer queries by greedy graph traversal [26], and a graph engine evaluates a row of M directly by local forward-push with total work $O(1/(\alpha\varepsilon))$ independent of catalog size [5]. From this single structural difference — *factorize once and read the factors* versus *never factorize and read the kernel at query time* — we derive three operational consequences for recommendation: (i) a cost model with $O(1)$ ingestion of interaction events and size-independent local reads; (ii) cold-start exactness at any sample size, below the matrix-completion threshold where low-rank recovery is information-theoretically impossible [29, 30, 31]; (iii) native non-stationarity, where TTL-decayed edge weights make the kernel a function of time that is evaluated at read time, while a factorization serves a frozen snapshot. Each claim is supported by an established strand of the recommender-systems literature: random-walk recommenders match or beat factorization models under proper tuning [50, 51, 64], nearest-neighbor and co-occurrence methods remain state-of-the-art for playlist continuation and session-based music recommendation [57, 58, 60, 62], and constant-time random walks serve real-time recommendation at three-billion-node scale in production [52]. We add one controlled replication (§8.1): under the exact Mult-VAE/EASE evaluation protocol on the Million Song Dataset [77, 47], tuned discounted walk readers reach NDCG@100 = 0.323 versus Mult-VAE’s published 0.316 (Recall@20 0.262 vs 0.266) — parity with the neural state of the art of that comparison, from a reader with no training stage at all — while full-rank EASE remains ahead at 0.389, consistent with our rank-obstruction analysis. We also state the honest converses: low-rank completion is a generalization capability that direct evaluation deliberately lacks (§6.5), and audio-content encoders solve the *zero*-interaction cold start that no interaction graph can (§7.2).

1. Introduction

Music platforms observe a stream of timestamped interaction events — plays, skips, playlist insertions, track-to-track transitions within sessions — and must answer proximity queries over a catalog V of items: *given this track, this session, or this playlist prefix, rank the candidates for what comes next*. Automatic playlist continuation [60, 61] and session-based next-track recommendation [55, 57, 59] are the canonical forms of this task, and both sit on the standing challenge list of the field [71].

Two architectures dominate:

- **Embedding + ANN.** Learn an encoder $\varphi : V \rightarrow \mathbb{R}^d$ from the interaction stream — item2vec-style skip-gram over listening sessions [53, 54], matrix factorization over the user–item matrix [74, 75, 76], or a content encoder over audio [69] — index $\{\varphi(v)\}$ in an ANN structure, and answer a query q with $\arg \text{top-}k_v \langle \varphi(q), \varphi(v) \rangle$.
- **Graph-based retrieval.** Maintain an explicit weighted directed graph on V whose edges aggregate the same events, and answer with a bounded traversal from the query vertex: a truncated beam-pruned walk, Personalized PageRank [3, 4, 5], or a local community around the seed. This lineage runs from item-to-item collaborative filtering [43, 44, 45] through ItemRank [48], commute-time kernels [9], P_α^3 and RP_β^3 [49, 50, 51], to production systems such as Pinterest’s Pixie [52].

These are usually treated as different philosophies — “latent semantics” versus “neighborhood heuristics”. The first contribution of this paper is to collapse the distinction: both architectures are **readers of one object**, a walk kernel $M = \sum_{k \geq 0} c_k P^k$ built from the transition operator P of the interaction graph. The identification is not an analogy:

1. The flagship embedding methods are (implicit or explicit) **low-rank factorizations of walk kernels** — a sequence of theorems, not a heuristic correspondence (§3; [12, 22, 21]). In particular, item2vec trained on listening sessions factorizes the shifted PMI of exactly the co-occurrence counts that the interaction graph stores as edge weights (Corollary 3.3).
2. The graph reader evaluates a row of the kernel **directly, locally, at query time**, with a certified work bound $O(1/(\alpha\varepsilon))$ independent of $|V|$ and $|E|$ (§4; [5]).
3. Production ANN search is itself greedy graph traversal [26, 27, 28], so each architecture literally implements the other’s search procedure (Proposition 3.6).

The second contribution is to derive, from the single structural difference between the readers — the presence or absence of a factorization stage — three operational claims that practitioners observe but rarely state together, and to give each its precise mathematical form:

- **C1 (complexity, §7.1).** Ingesting one interaction event is an $O(1)$ local graph update versus an encoder forward pass plus index insert; a query is a size-independent local push versus a query-encoding forward pass plus $\Theta(d)$ -per-comparison ANN search; and the factorized pipeline carries a refresh stage (retrain + re-embed + re-index) that the direct pipeline does not possess at all.
- **C2 (cold start, §7.2).** The direct reader’s guarantees are sample-size-free: a track is retrievable, with exact asserted proximity, from its first observed interaction. The factorized reader is a statistical estimator with a sample-complexity threshold of order $nr \text{polylog}(n)$ observed entries [29, 30, 31], below which the kernel it serves is non-identifiable — the precise form of the familiar new-release problem [42, 69].
- **C3 (non-stationarity, §7.3).** With TTL-decaying edge weights the kernel is a right-continuous function of time evaluated at the query’s own instant; forgetting is the resting state of the store, not an operation. A factorization serves $\widehat{M} \approx M(t_0)$ and its error grows with the drift $\|M(t) - M(t_0)\|$ until the global refresh cost is paid again — a serious constraint in a domain governed by release cycles, charts, and virality [66, 67, 70].

§8 assembles the empirical record the recommender-systems community has already produced — reproducibility studies [64, 65], session-based evaluations on music corpora [56, 57, 58], the ACM RecSys Challenge 2018 on playlist continuation [61, 62, 63], full-rank versus low-rank linear models [46, 47], and the Pixie production system [52] — and shows that it is consistent with C1–C3. We contribute one new

data point ourselves: a replication of the Liang et al. protocol on the Million Song Dataset with the graph readers of §4–§5 (§8.1), calibrated against the published baselines on the identical split. Throughout, mathematics is kept at the level of precise statements with short proofs where they are genuinely short, and citations where they are not.

2. Setting: interaction streams, decaying graphs, walk kernels

2.1 The stream and the graphs it induces

Fix a finite item set V (tracks; the constructions below apply verbatim to artists, albums, or playlists). The platform observes events (u_j, v_j, t_j) — “ u_j and v_j co-occurred at time t_j ” — where co-occurrence may mean: played within one session window, adjacent in a playlist, or a direct listening transition $u \rightarrow v$. Two standard graphs carry this information:

- the **bipartite interaction graph** B on $\text{users} \cup \text{items}$, with an edge per play; and
- the **item–item projection** G on V , with $w(u, v)$ aggregating co-occurrence evidence.

The two are interconvertible at the level of walks: if P_B is the transition matrix of B , then $(P_B^2)_{i,i'} = \sum_u P_B(i, u) P_B(u, i')$ is a degree-normalized co-play count, so one hop on the projection corresponds to two hops on the bipartite graph, and the classical P^3 recommender (§2.4) is a one-hop neighborhood on the projection composed with the user’s own profile. We work on the projection G ; every statement transfers to B with hop counts doubled.

2.2 Decaying edge weights

Definition 2.1 (decaying weight). An edge (u, v) carries a finite multiset of contributions $\{(c_i, e_i)\}_i$, one per observed event, with values $c_i \in \mathbb{R}$ and expirations $e_i \in \mathbb{R}$. Its weight at time t is

$$w_t(u, v) = \sum_i c_i \mathbf{1}\{t < e_i\}.$$

Thus $t \mapsto w_t(u, v)$ is a right-continuous step function decaying to 0 after the last expiration: each entry of the weight matrix is a **sliding-window sum of evidence**, a time-decaying stream aggregate in the sense of Cohen–Strauss [41]. Ingesting one event appends one contribution — an $O(1)$ local operation — and a read evaluates the sum at a single instant chosen once per traversal. The graph at time t is $G_t = (V, E_t, w_t)$ with $E_t = \{(u, v) : w_t(u, v) \neq 0\}$, directed and weighted, with no other edge decoration.

In the music setting Definition 2.1 says: an edge is the statement “these two tracks were co-listened, with this much evidence, within the trailing window”, where the window length (the TTL) is the operator’s choice of how long taste evidence stays relevant. Directionality is retained because listening transitions are ordered: the probability of v following u in a session is not that of u following v (§6.3).

2.3 Reweighting and the transition operator

Raw co-occurrence counts overweight globally popular tracks — the popularity bias endemic to music data, where item popularity is heavy-tailed [70]. Before any traversal we therefore allow an entrywise re-scoring of weights: with $\text{df}(v)$ the in-degree of v over distinct tails,

$$\psi_{\text{raw}}(w) = w, \quad \psi_{\text{tfidf}}(w \mid v) = \frac{w}{\log_2(1 + \text{df}(v))}, \quad \psi_{\text{bm25}}(w \mid v) = \text{IDF}(\text{df}(v)) \cdot \frac{w(k_1 + 1)}{w + k_1(1 - b + b \frac{|u|}{|v|})},$$

the last being Okapi BM25 [17] with term frequency = the live edge weight and document length $|u| = \text{df}(u)$ the tail’s out-degree. §5 shows this is the same correction the embedding and random-walk-recommender

literatures apply under the names *PMI shift* and *β -popularity-discount*, respectively. Write A_t^ψ for the reweighted adjacency matrix, $d^\psi(u) = \sum_v A_t^\psi(u, v)$, and define the row-substochastic transition matrix

$$P(u, v) = \begin{cases} A_t^\psi(u, v) / d^\psi(u) & d^\psi(u) > 0, \\ 0 & \text{otherwise (sinks: the row is zero and walk mass dies).} \end{cases}$$

2.4 Walk kernels

Definition 2.2 (walk kernel). Given nonnegative coefficients (c_k) with $\sum_k c_k < \infty$, the associated kernel is

$$M = \sum_{k \geq 0} c_k P^k.$$

$M(u, v)$ aggregates the probability of reaching v from u over walks of every length, discounted by c_k . The members relevant here:

Kernel	Coefficients	Closed form / recommender
Katz index [1]	$c_k = \beta^k$ on A rather than P	$(I - \beta A)^{-1} - I$, $\beta < 1/\rho(A)$
Personalized PageRank [2, 3, 4]	$c_k = \alpha(1 - \alpha)^k$	$\alpha(I - (1 - \alpha)P)^{-1}$; ItemRank [48], Pixie [52]
P^3 family [49, 50]	$c_3 = 1$, else 0 (on B)	P_B^3 ; with discounts: P_α^3 , RP_β^3 (§5)
Heat / diffusion kernel [8]	$c_k = e^{-\beta} \beta^k / k!$	$e^{-\beta(I-P)}$
DeepWalk window kernel [22]	$c_k = \frac{1}{T} \mathbf{1}\{1 \leq k \leq T\}$	$\frac{1}{T} \sum_{r=1}^T P^r$; the kernel item2vec factorizes (§3)

Proposition 2.3 (PPR is well defined). For $\alpha \in (0, 1)$ the series $\alpha \sum_{k \geq 0} (1 - \alpha)^k P^k$ converges and equals $\alpha(I - (1 - \alpha)P)^{-1}$.

Proof. P is substochastic, so $\|P\|_\infty \leq 1$ and $\|(1 - \alpha)P\|_\infty \leq 1 - \alpha < 1$; the Neumann series of $I - (1 - \alpha)P$ converges in operator norm. $\square$

We write $\pi_s = \alpha e_s^\top (I - (1 - \alpha)P)^{-1}$ for the PPR row seeded at s ; equivalently π_s is the unique fixed point of $\pi_s = \alpha e_s^\top + (1 - \alpha)\pi_s P$. Probabilistically, $\pi_s(v)$ is the stationary mass at v of a listener-surrogate who restarts at the seed track with probability α at each step; $\|\pi_s\|_1 \leq 1$, with equality iff no mass dies in sinks.

2.5 The task

Every recommender considered in this paper computes, for a seed s (current track, session representative, or playlist prefix),

$$\arg \text{top-} k_{v \in V} M(s, v)$$

for some walk kernel M — the only question is *how the row $M(s, \cdot)$ is read*. §3 and §4 describe the two readers.

3. Reader I: factorize, then read

The embedding pipeline is: (i) learn or compute $\Phi \in \mathbb{R}^{|V| \times d}$; (ii) index the rows; (iii) at query time return top- k inner products. Step (iii) reads a row of $\Phi\Phi^\top$. The content of this section is that for the training objectives actually used in music recommendation, $\Phi\Phi^\top$ is a low-rank approximation of a walk kernel over the interaction graph of §2 — so the pipeline as a whole reads a row of $\widehat{M} \approx f(M)$.

Theorem 3.1 (Levy–Goldberg [12]). Skip-gram with negative sampling (SGNS; word2vec [11]) with b negative samples, trained to its global optimum with unconstrained dimension, produces item/context vectors whose inner products satisfy

$$\langle \varphi(u), \varphi_c(v) \rangle = \text{PMI}(u, v) - \log b, \quad \text{PMI}(u, v) = \log \frac{\text{Pr}(u, v)}{\text{Pr}(u) \text{Pr}(v)},$$

where Pr is the co-occurrence distribution of the training corpus. That is, SGNS implicitly factorizes the shifted pointwise-mutual-information matrix [15]; with $d < |V|$ it computes a rank- d approximation of it. GloVe [13] makes the log-co-occurrence factorization explicit, and Arora et al. [14] derive the PMI identity from a generative model.

Remark 3.2. item2vec [53] and prod2vec [54] are precisely SGNS run on interaction sequences — listening sessions, playlists, purchase streams — in place of sentences. Theorem 3.1 therefore applies verbatim.

Corollary 3.3 (shared sufficient statistics). Fix an event stream and a co-occurrence window. Let $\#(u, v)$ be the windowed co-occurrence counts, $\#(u), \#(v)$ the marginals, $D = \sum_{u,v} \#(u, v)$. Then:

1. item2vec trained on this stream implicitly factorizes $\log(\#(u, v) D) - \log(\#(u)\#(v)) - \log b$;
2. the graph of §2 built from the same stream stores $w(u, v) \propto \#(u, v)$ (exactly, when contributions are unit and the TTL window equals the corpus horizon).

Hence the two systems are estimators built on the *same sufficient statistics*; they differ only in the transform applied (log and marginal shift versus ψ of §2.3 — see §5) and in whether a rank- d bottleneck sits between the statistics and the reader.

Proof. (1) is Theorem 3.1 with the empirical distribution $\text{Pr}(u, v) = \#(u, v)/D$. (2) is Definition 2.1 with $c_i \equiv 1$. $\square$

Theorem 3.4 (NetMF; Qiu et al. [22]). DeepWalk [18] with window T and b negative samples, in the limit of an infinite random-walk corpus, implicitly factorizes the elementwise logarithm of the positive part of

$$\frac{\text{vol}(G)}{bT} \left(\sum_{r=1}^T P^r \right) D^{-1},$$

where $\text{vol}(G) = \sum_{u,v} A(u, v)$ and D is the degree matrix. LINE [19] is the case $T = 1$; node2vec [20] factorizes the analogous expression for its second-order walk. In words: **graph-embedding methods are rank- d factorizations of (a log transform of) the DeepWalk window kernel of Definition 2.2** — the same object the P^3 recommenders of §5 evaluate directly.

Explicit constructions. HOPE [21] skips the sampling and directly factorizes kernels $S = M_g^{-1} M_l$ — Katz ($M_g = I - \beta A$, $M_l = \beta A$) and rooted PageRank (the PPR resolvent) among them — into asymmetric source/target factors $S \approx U^s (U^t)^\top$ via partial generalized SVD. VERSE [23] trains embeddings whose inner-product softmax matches PPR rows; InstantEmbedding [24] computes a single item’s embedding from one local PPR push plus hashing — literally a compression of π_s into $\mathbb{R}^d$. Laplacian eigenmaps [25] is the spectral ancestor. On the matrix-factorization side of recommendation, WRMF [75] and BPR [76] factorize (reweightings of) the bipartite interaction matrix itself — the B -side counterpart of the same construction [74].

Remark 3.5 (systematic error). For a symmetric kernel the best rank- d approximation in Frobenius or spectral norm is the truncated eigendecomposition (Eckart–Young [73]), with error exactly the discarded spectral tail $(\sum_{i>d} \sigma_i^2)^{1/2}$. The factorized reader’s systematic error is therefore governed by the spectral decay of M ; the ANN index adds a recall error on top.

The converse: ANN search is graph traversal. HNSW [26] and its navigable-small-world ancestor [27] answer nearest-neighbor queries by greedy beam search over a proximity graph constructed from the learned metric; theory for the family is developed in [28].

Proposition 3.6 (exact simulation on k NN graphs). Let G_φ be the directed graph with an edge $u \rightarrow v$ of weight $\langle \varphi(u), \varphi(v) \rangle$ (shifted positive) whenever v is among the k exact nearest neighbors of u . Then a one-hop, beam- k , largest-weight traversal from seed s on G_φ returns exactly the vector-search top- k of s .

Proof. Immediate: the frontier of the seed is its stored k NN list, ranked by the stored weights. $\square$

So graph traversal subsumes vector retrieval given the right graph, and practical vector retrieval *is* approximate graph traversal. The question “graph or vectors?” is not about the search procedure; it is about **where the graph comes from** — accumulated from observed interactions as ground truth, or reconstructed from a learned metric — and about whether a factorization stage sits between the data and the reader.

4. Reader II: read the kernel directly

The direct reader maintains G_t of §2 and serves three traversal operators, in kernel language:

- **Truncated beam walk** (step hops, beam k): a greedy, per-hop top- k -pruned reader of $\sum_{r \leq \text{step}} P^r$ -type locality — algorithmically the same beam search HNSW runs (Proposition 3.6), executed on the asserted graph. This is the item-to-item CF of [43, 44, 45] with hops.
- **Personalized PageRank**: a certified reader of one row of $\alpha(I - (1 - \alpha)P)^{-1}$ via Andersen–Chung–Lang (ACL) forward-push [5] (bookmark-coloring in Berkhin’s terminology [6]); Monte-Carlo alternatives in [72, 52].
- **Local community**: PageRank-Nibble [5] — the same push followed by a sweep cut minimizing conductance, returning a cohesive listening neighborhood (a micro-genre or scene around the seed) rather than a ranked list.

The push is where the mathematics is sharpest. The algorithm maintains $p, r : V \rightarrow \mathbb{R}_{\geq 0}$ with $p = 0$, $r = e_s$ initially, and repeatedly picks any u with $r(u) \geq \varepsilon \cdot \max(\deg(u), 1)$ (deg counting live out-edges) and **pushes**:

$$p(u) \leftarrow p(u) + \alpha r(u), \quad r(v) \leftarrow (1 - \alpha) r(u) P(u, v) \quad \forall v, \quad r(u) \leftarrow 0.$$

It terminates when no vertex exceeds its threshold and returns p as the estimate of π_s .

Lemma 4.1 (invariant). At every step of the algorithm,

$$\pi_s = p + \sum_{u \in V} r(u) \pi_u.$$

Proof. Initially $p = 0$, $r = e_s$, and the identity reads $\pi_s = \pi_s$. A push at u with residual $\rho = r(u)$ replaces the term $\rho \pi_u$ by $\alpha \rho e_u + (1 - \alpha) \rho \sum_v P(u, v) \pi_v$ — the first summand entering p , the second entering the residuals. By the fixed-point identity $\pi_u = \alpha e_u + (1 - \alpha) \sum_v P(u, v) \pi_v$, the right-hand side is unchanged. $\square$

Since every $\pi_u \geq 0$, Lemma 4.1 gives $0 \leq p \leq \pi_s$ entrywise: the estimate never overshoots, and the error has the exact representation $\pi_s - p = \sum_u r(u) \pi_u$.

Theorem 4.2 (work bound [5]). With threshold $\varepsilon \cdot \max(\deg(u), 1)$, the number of pushes is at most $1/(\alpha\varepsilon)$, and moreover $\sum_{\text{pushes}} \deg(u_i) \leq 1/(\alpha\varepsilon)$, so the total work is $O(1/(\alpha\varepsilon))$ — **independent of $|V|$ and $|E|$** , i.e., of catalog and audience size.

Proof. A push at u with residual ρ changes $\|r\|_1$ by $-\rho + (1-\alpha)\rho \sum_v P(u, v) \leq -\alpha\rho$, using substochasticity of the row. Since $\|r\|_1 = 1$ initially and $\|r\|_1 \geq 0$ always, and each push has $\rho \geq \varepsilon \max(\deg(u), 1) \geq \varepsilon$, the number of pushes is at most $1/(\alpha\varepsilon)$; summing $\alpha\varepsilon \deg(u_i) \leq \alpha\rho_i$ over pushes gives $\sum_i \deg(u_i) \leq 1/(\alpha\varepsilon)$, which dominates the work since a push at u costs $O(\deg(u))$. $\square$

Theorem 4.3 (approximation, undirected case [5]). If the graph is undirected (w symmetric, ψ symmetry-preserving), then at termination

$$0 \leq \pi_s(v) - p(v) \leq \varepsilon d(v) \quad \text{for every } v.$$

Proof. For reversible $P = D^{-1}A$ with symmetric A , the matrix $DP^k = (AD^{-1})^{k-1}A$ is symmetric for every $k \geq 1$, hence so is $D\Pi$ for the PPR kernel Π ; entrywise this is the reciprocity identity $d(u)\pi_u(v) = d(v)\pi_v(u)$. By Lemma 4.1 and the termination condition $r(u) \leq \varepsilon d(u)$,

$$\pi_s(v) - p(v) = \sum_u r(u)\pi_u(v) \leq \varepsilon \sum_u d(u)\pi_u(v) = \varepsilon d(v) \sum_u \pi_v(u) \leq \varepsilon d(v). \quad \square$$

Remark 4.4 (directed case). Listening-transition graphs are directed, and reciprocity fails there. Lemma 4.1 and Theorem 4.2 use only linearity and substochasticity and hold verbatim: the estimate remains a certified underestimate with exact error representation and size-independent cost. The clean entrywise bound of Theorem 4.3 is specific to reversible chains and should be treated as a heuristic in the directed setting. Sinks (tracks with no live out-edges) absorb the restart share of their residual and leak the continuation share — exactly the substochastic P of §2.3, so no separate analysis is needed.

Sweep cut. The local-community operator orders the push’s support by $p(v)/\max(d^\psi(v), 1)$ and returns the prefix minimizing the directed, weighted, out-volume conductance

$$\phi(S) = \frac{\text{cut}(S)}{\min(\text{vol}(S), \text{vol}(\text{touched}) - \text{vol}(S))}$$

— PageRank-Nibble. For undirected graphs ACL prove, via the Lovász–Simonovits curve [7], that if the seed lies well inside a set of conductance ϕ , the sweep finds a set of conductance $O(\sqrt{\phi \log n})$ with corresponding locality of work [5]. As with Theorem 4.3, the directed weighted case inherits the *procedure* with exact invariants and the *guarantee* on the symmetric restriction.

Production precedent. These are not paper algorithms. Pixie [52] serves recommendations at Pinterest by biased random walks with early stopping over an object graph of 3 billion nodes and 17 billion edges; a single server sustains $\sim 1,200$ requests/second at ~ 60 ms latency, the walk runs in constant time *independent of graph size* (the Monte-Carlo counterpart of Theorem 4.2), and graph pruning — their analogue of §2.3’s reweighting — improved recommendation quality by a further 58%. Real-time, catalog-scale direct evaluation is an engineering fact.

5. Hub suppression: one correction, three literatures

Why re-score weights before walking (§2.3)? For the same reason three literatures independently converged on the same correction: raw evidence is dominated by globally popular items, and dividing out an occurrence prior converts *popular* into *informative*.

- **Lexical statistics / embeddings.** SGNS factorizes PMI, not raw counts (Theorem 3.1). Writing the empirical PMI as

$$\text{PMI}(u, v) = \log \#(u, v) - \log \#(u) - \log \#(v) + \log D,$$

the marginal subtractions are a popularity discount: at fixed marginals PMI is monotone in the co-occurrence count, and candidates are re-ranked *only* through their popularity terms [12, 15].

- **Information retrieval.** IDF [16] and BM25 [17] are the same discount for term evidence; §2.3’s $\psi_{\text{tfidf}}, \psi_{\text{bm25}}$ apply them to edge evidence, with df counted over distinct tails.
- **Random-walk recommenders.** P_α^3 [49] sharpens transition probabilities elementwise, $(P)_{uv} \mapsto (P)_{uv}^\alpha$, and RP_β^3 [50, 51] divides the 3-hop score by an item-degree power:

$$S_{\text{RP}_\beta^3}(u, i) = \frac{(P_B^3)_{u,i}}{d(i)^\beta}.$$

The β -discount is what lifted the plain P^3 walk to state-of-the-art accuracy *and* long-tail coverage on standard benchmarks [50] — and it is, structurally, the PMI marginal subtraction after exponentiation.

Remark 5.1. All three corrections are entrywise transforms of the kernel’s input applied *before* the reader runs, of the common shape evidence $\mapsto$ evidence / popularity prior. The preprocessing that makes factorization meaningful and the preprocessing that makes traversal meaningful are the same operation. This matters for the equivalence question: when comparing the two readers one should compare them on the *same* transformed kernel, and the music-recommendation literature’s best walk methods and best embedding methods already do apply matched corrections.

6. When the two readers agree — and when they cannot

6.1 The agreement regime

Proposition 6.1. Fix a kernel M . If (i) M is symmetric positive semidefinite of rank at most d , so $M = \Phi\Phi^\top$ exactly, and (ii) the ANN index has perfect recall, then the factorized and direct readers return identical top- k sets for every seed.

Proof. Under (i) the inner products $\langle \varphi(u), \varphi(v) \rangle$ equal $M(u, v)$ exactly; under (ii) the index returns the exact top- k of that row, which is the direct reader’s output by definition (§2.5). $\square$

Perturbing (i) costs the spectral-tail error of Remark 3.5; perturbing (ii) costs index recall. So the two readers provably agree on near-low-rank, near-symmetric kernels — which is *why* embedding methods work at all, and why proximity indices and latent-factor models have traded blows on link prediction and recommendation since [10, 9]. The meaningful comparisons live outside that regime.

6.2 Obstruction: rank

Seshadhri, Sharma, Stolman, and Goel [32] prove that in the standard dot-product model, low-dimensional factorizations cannot reproduce the combination of sparsity (low average degree) and triangle density characteristic of real interaction networks: any $d \ll \text{polylog } n$ loses either the degree distribution or the triangle count. Co-listening graphs are exactly such networks — heavy-tailed degrees with dense local clustering around scenes and genres [70]. Chanpuriya et al. [33] sharpen the picture: with asymmetric factors and a logistic link, exact low-rank representations of sparse graphs exist, so the obstruction is partly model-specific. The fair summary: **whether the interaction kernel is representable in $\mathbb{R}^d$ is a delicate, model-dependent question with explicit impossibility theorems for the natural inner-product model; the graph store carries the kernel exactly and is indifferent to it.**

6.3 Obstruction: asymmetry and non-metric structure

$\langle \varphi(u), \varphi(v) \rangle$ is symmetric. Musical succession is not: an interlude leads into the album’s hit far more often than the reverse; a canonical recording leads to its covers differently than covers lead back. Sequence-aware evaluations quantify how much of the signal in music sessions is directional [59, 57]. Asymmetric factorizations ($U^s(U^t)^\top$, HOPE [21]; word2vec’s input/output vectors) recover directionality at the price of doubled parameters and the loss of the metric structure ANN indexes exploit. Non-metric, asymmetric similarity is moreover the *empirically observed* shape of human similarity judgment (Tversky [34]); Gaussian and hyperbolic embeddings [35, 36] exist precisely because flat inner products fail on hierarchical data — and genre taxonomies are hierarchical.

6.4 Obstruction: composition

A walk kernel composes multiplicatively along paths: two strong hops yield a quantifiable two-hop score, and $M(u, v) = 0$ exactly when no path carries weight. Inner-product similarity does not compose: $\text{sim}(a, b)$ and $\text{sim}(b, c)$ constrain $\text{sim}(a, c)$ only weakly. The graph can assert “ $a \sim b, b \sim c, a \not\sim c$ ” at full sharpness — a producer collaborates across two scenes that share no audience — while any rank- d factorization smooths it.

6.5 Completion versus fidelity

A rank- d factorization fitted to observed entries *fills in* the unobserved ones — the mechanism of matrix completion [29, 31] — and this is exactly why embedding retrieval generalizes: it surfaces similar-but-never-co-listened tracks. Direct evaluation returns 0 for pairs with no connecting path: no invented similarity, and no serendipity either.

The same theory prices the generalization. Exact (and stable) recovery of an incoherent rank- r kernel on n items requires on the order of $nr \text{polylog}(n)$ observed entries, and $\Omega(nr \log n)$ is information-theoretically necessary [30]. Below the threshold the factorization is not merely inaccurate — the missing entries are *non-identifiable*. This is the precise form of the cold-start claim C2, to which we now return.

7. The three operational consequences

All three follow from one structural fact: **the factorized pipeline contains a factorization stage and the direct pipeline does not.** The stage is (a) a global computation, (b) a statistical estimator with a sample-complexity threshold, and (c) a snapshot taken at a time t_0 . Deleting the stage deletes the three costs; the price is §6.5’s generalization.

7.1 C1: complexity

	Direct reader (graph)	Factorized reader (embedding + ANN)
Ingest one play / co-listen event	append one contribution: amortized $O(1)$, local (Def. 2.1)	encoder forward pass ($O(\text{model})$, accelerator-bound) + ANN insert ($O(d \cdot \text{ef} \cdot \log n)$ empirically [26])
Query (next-track / continuation)	push: total work $\leq 1/(\alpha\varepsilon)$, independent of V , E (Thm 4.2); comparisons are scalar reads	encode the query (one forward pass) + ANN search (empirically $O(d \text{ef} \log n)$ [26, 28]); every comparison costs $\Theta(d)$, $d \in [64, 3072]$ typical
Model refresh under drift	the stage does not exist	retrain / re-encode + re-index: global, $\Omega(n)$

	Direct reader (graph)	Factorized reader (embedding + ANN)
Delete / forget one item’s influence	wait for expiry, or delete the edge: $O(1)$	unlearning problem [39, 40]; streaming ANN deletion needs periodic consolidation [38]
Steady-state memory	$O(V + E_t)$, TTL decay actively shrinking E_t	$n \cdot d$ floats + index graph overhead

Two honesty notes. First, a warm ANN index answers queries in empirically logarithmic time with excellent recall [26]; the categorical gap is not the query asymptotics but (i) the $\Theta(d)$ -versus- $\Theta(1)$ comparison cost, (ii) the encoder forward pass on the query path, which typically dominates end-to-end latency, and (iii) the existence of the refresh row at all. Second, a factorized system amortizes training over queries: for a *static* catalog at very high query volume it can win on total cost. The comparison tilts toward the direct reader in proportion to the write/query ratio and the drift rate — and music platforms sit at high write rates and high drift.

7.2 C2: cold start

Proposition 7.1 (sample-size-free exactness). For any graph — including one with a single edge — Lemma 4.1 and Theorem 4.2 hold as stated; in particular an item is retrievable from the instant its first interaction is ingested, with proximity equal to the asserted evidence (exactly, at one hop). No hypothesis on $|E|$, the weight distribution, or a generative model is used anywhere in §4.

Contrast the factorized reader at both of its levels:

- **Corpus level.** Below $\Theta(nr \text{ polylog } n)$ observed interactions the low-rank kernel is non-identifiable (§6.5; [29, 30, 31]); a factorization fitted there returns noise with confident geometry. A new platform, a new market, or a niche catalog segment lives in this regime.
- **Item level.** An interaction-trained embedding (item2vec, WRMF, BPR) is undefined for a track with no interactions — the classical new-item problem [42]. In music this is acute: catalogs grow by tens of thousands of tracks daily, and the head–tail contrast is extreme [70].

One qualification is required for intellectual honesty, and it is music-specific. *Content* encoders sidestep the item-level problem for items that have content: van den Oord et al. [69] predict collaborative latent factors directly from audio, so a track with *zero* plays can be placed in the retrieval space. The direct reader has no answer at zero interactions — its first edge is its first information. The precise division is therefore: at $n_{\text{item}} = 0$ interactions, content embeddings are the only option; for $1 \leq n_{\text{item}} \ll$ the completion threshold, the direct reader is exact while interaction-trained factorizations are noise; both readers converge in the data-rich regime (§6.1). Entities defined *only* by their relations — users, sessions, playlists — never have content, and for them the graph’s advantage extends to the entire sparse regime.

7.3 C3: non-stationarity

Music consumption is aggressively non-stationary: release cycles, charts, seasonal listening, virality. The recommendation literature treats temporal dynamics as a first-class modeling problem — time-weighted CF [67], timeSVD++ [66], incremental factorization with forgetting [68] — but note the architectural position of these corrections: they reweight or extend the *training objective*. The served model is still a snapshot.

The direct reader needs no correction because its kernel is a function of time by construction: Definition 2.1 makes every entry of A_t a sliding-window sum, each traversal evaluates at a single instant, and forgetting is not an operation but the resting state ($O(1)$ per contribution, enforced by expiry). Formally, the direct reader computes top- k of $M(t)$ at the query’s own t ; the factorized reader serves $\widehat{M} \approx M(t_0)$, with retrieval

error against the live kernel growing with the drift $\|M(t) - M(t_0)\|$ until the factorization cost is paid again. Three compounding frictions are documented:

1. **Refresh is global.** Incremental encoder updates or index inserts do not update the *kernel estimate*; only retraining does, at $\Omega(n)$.
2. **Embedding spaces are unstable across refreshes.** Retrained embeddings agree with their predecessors only up to (at best) an orthogonal transformation, which must itself be estimated (Procrustes alignment [37]); vectors from different runs are not directly comparable, so a refresh forces a full re-encode and re-index rather than a rolling one.
3. **Deletion is adversarial to learned representations.** Removing one item’s or user’s influence from trained parameters is the machine-unlearning problem [39, 40] — increasingly a legal requirement, not a preference — and even at the index level, high-churn deletion in ANN structures requires consolidation [38]. In the direct reader, deletion is the default: evidence that is not re-asserted expires.

Scope. Decay is the correct prior for *evidential*, *event-sourced* relations (“these tracks were co-listened this month”), which age and should. It is the wrong prior for *stationary* similarity (“this recording is a cover of that song”), which does not decay and which a content encoder captures zero-shot. The two readers are optimized for different stationarity regimes; §10 returns to their composition.

8. Evidence from the recommendation literature

The claims of §6–§7 are consilient with an unusually broad published record (§8.2); we first add one controlled measurement of our own (§8.1).

8.1 A replication on the Million Song Dataset

We reproduce the evaluation protocol of Liang et al. [77] on the Million Song Dataset taste-profile subset — the split on which Mult-VAE [77] and EASE [47] publish their music-domain numbers — and evaluate the direct readers of §4 with the §5 discounts on it. Protocol, faithful to the reference implementation (including its random seeds): songs with ≥ 200 listeners, then users with ≥ 20 songs, giving 571,355 users, 41,140 items, 33.6M interactions (0.14% dense — our filtered statistics match [77, Table 1] exactly); 50,000 validation and 50,000 test users held out entirely from training (strong generalization); per heldout user, 80% of items form the fold-in query profile and metrics are computed on the remaining 20%; Recall@20/50 (normalized by $\min(K, |\text{heldout}|)$) and NDCG@100, averaged over users. All hyperparameters are selected on the validation users only; test users are scored once. Scripts: [experiments/msd-walk-kernels](#).

The graph readers are built from the training matrix alone: the P^3 projection kernel $W = P_{iu}P_{ui}$ (top-1000 per row), its RP_β^3 column discount $\text{pop}^{-\beta}$, item-kNN cosine, and the truncated PPR mixture $\sum_{k \leq 3} a(1 - a)^{k-1} \hat{x}P_1^k$ on the row-normalized kernel with the same $\text{pop}^{-\beta}$ discount — the §4 direct reader with the §5 correction. Our popularity baseline reproduces the published one to three decimals (0.043/0.068/0.058), calibrating the pipeline before any comparison is made.

Reader	Recall@20	Recall@50	NDCG@100
<i>published on the identical split [77, 47]</i>			
Popularity	0.043	0.068	0.058
CDAE	0.188	0.283	0.237
WMF (low-rank MF)	0.211	0.312	0.257
SLIM	— did not finish —		
Mult-DAE	0.266	0.363	0.313
Mult-VAE ^{PR}	0.266	0.364	0.316

Reader	Recall@20	Recall@50	NDCG@100
EASE (full-rank item-item) <i>measured, this work</i> (<i>graph readers</i>)	0.333	0.428	0.389
Popularity (calibration)	0.043	0.068	0.058
P^3 ($\alpha=1$, $K=200$)	0.176	0.269	0.227
item-kNN cosine ($K=1000$)	0.236	0.319	0.291
RP_β^3 ($\beta=0.4$)	0.262	0.362	0.323
PPR, discounted ($a=0.9$, $\beta=0.4$, $k \leq 3$)	0.262	0.362	0.323

Four readings, in decreasing order of confidence:

1. **The discounted walk readers match the neural state of the art of this comparison.** RP_β^3 and the discounted PPR reader sit at $NDCG@100 = 0.323$ against Mult-VAE's 0.316 and Mult-DAE's 0.313, with Recall@20/50 within 0.004/0.002 of Mult-VAE — from a reader with no training stage, no encoder, $O(1)$ event ingestion, and a kernel that §7.3's argument keeps live under drift. They exceed the low-rank factorization WMF by +26% NDCG@100 and CDAE by +36% — on the very protocol those models were tuned for, in the data-rich regime most favorable to factorization (the average heldout user folds in ≈ 48 items; cold start and drift, where §7 predicts the gap widens, are not probed here at all).
2. **Hub suppression is worth +42% NDCG@100** ($0.227 \rightarrow 0.323$): the β discount, not walk depth, separates a mediocre graph reader from a state-of-the-art-matching one — §5 measured. Undiscounted multi-hop mixtures *degrade* with depth (popularity contamination compounds per hop), while after the discount the $k \leq 3$ mixture at $a = 0.9$ edges the one-hop reader by a hair on validation and ties it on test: with ~ 48 fold-in seeds per user, one discounted hop already spans the relevant neighborhood. Multi-hop locality should matter precisely where this protocol has no coverage — few-seed (cold-start) queries.
3. **EASE's lead (0.389) is the rank story, not the factorization story.** EASE is itself a one-hop item-item kernel — full-rank, with the reweighting *learned* by a global ridge solve rather than fixed heuristically as in §2.3/§5 — and it beats every low-rank model by a larger margin than it beats our fixed- ψ readers. That is §6.2's prediction (the rank- d bottleneck costs accuracy) read empirically. The gap between EASE and RP_β^3 prices exactly one thing: learning the reweighting. It costs EASE the full factorized pipeline operationally — a global $O(|V|^{2.376})$ -ish solve as the refresh stage, a frozen snapshot under drift, and the unlearning problem on deletion — so C1–C3 apply to it unchanged; a learned-yet-locally-updatable ψ is an open direction (§10).
4. The absolute numbers carry the usual caveat of a single dataset and our own hyperparameter grid; the *relative* placement is protected by the exact protocol match and the popularity calibration.

8.2 The published record

Random-walk readers match or beat factorizations. P_α^3 and RP_β^3 — three-hop walk scores with the §5 discounts — matched or outperformed tuned matrix-factorization models on standard benchmarks while producing better long-tail coverage, at interactive update latencies [49, 50, 51]. In the reproducibility study of Ferrari Dacrema, Cremonesi, and Jannach [64] (Best Paper, RecSys 2019), of 18 recent neural recommenders only 7 could be reproduced with reasonable effort, and 6 of those 7 were outperformed by properly tuned simple baselines — nearest neighbors and precisely these graph walks. Rendle et al. [65] reached the same verdict for the dot-product versus learned-similarity question.

Full-rank beats low-rank when it is allowed to. EASE [47] — a full-rank, closed-form linear item-item model, i.e., a *one-hop kernel with a learned reweighting* and no rank bottleneck — outperforms deep

and low-rank models on most public implicit-feedback benchmarks, in line with SLIM [46] before it. This is §6.2 read empirically: the rank- d bottleneck is not a free compression.

Music specifically: neighborhoods and co-occurrence are the frontier. Bonnin and Jannach’s survey of playlist generation [60] found simple co-occurrence and popularity schemes rivaling elaborate models. Session-based evaluations on music logs [56, 57] found session- k NN variants matching or beating the RNN state of the art (GRU4Rec [55]). In the ACM RecSys Challenge 2018 on automatic playlist continuation over the Spotify Million Playlist Dataset [61], the organizers’ analysis [62] records that top teams were dominated by matrix factorization *plus* neighborhood/co-occurrence machinery; the winning system [63] used WRMF only to *generate* ~ 20 k candidates per playlist and produced its final ranking with gradient-boosted item-item and user-user neighborhood features — the factorized reader for recall, the direct kernel’s statistics for precision. A dedicated nearest-neighbor entry [58] placed among the leaders with a fraction of the machinery.

Direct evaluation at production scale. Pixie [52] (§4) demonstrates the cost model of §7.1 in production: constant-time walks on a 3-billion-node object graph, 60 ms end-to-end, real-time graph updates, no training stage between events and retrieval — and a measured +50% engagement over the batch-computed predecessor, i.e., over a stale kernel. This is C3 measured in the field.

None of this shows the direct reader dominates everywhere — §6.1 explains why both families work, and §6.5 why the factorized reader generalizes. What the record consistently refutes is the presumption that the factorized reader is the *default* and the graph reader the heuristic: on interaction data, the direct reader of the very kernel embeddings approximate is at least as accurate, radically cheaper to keep fresh, and exact in the sparse regimes where the factorization is undefined.

9. Reference implementation

The direct reader described in §2 and §4 is implemented, with the stated guarantees as design contracts, in **Lantern**, an in-memory graph key-vertex-store: Definition 2.1’s expiring contributions are the edge representation in [core/graphcache/edge.go](#); the beam walk, the ACL push (with the hard budget a constant multiple of Theorem 4.2’s $1/(\alpha\varepsilon)$), and the PageRank-Nibble sweep are **Neighbor**, **PersonalizedPageRank**, and **LocalCommunity** in [core/graphcache/traversal.go](#); the §2.3 reweightings are the **EdgeWeighting** enum, sharing one BM25 kernel with full-text search so that §5’s “one correction” is literally one function. Writes are amortized- $O(1)$ appends; every traversal evaluates at a single liveness instant, realizing “read $M(t)$ at the query’s own t ”.

10. Limitations and scope

- **Generalization is real.** The factorized reader retrieves similar-but-never-co-listened tracks; the direct reader cannot, by design (§6.5). Where interactions are abundant and stationary and serendipity is the product, embeddings are the right tool.
- **Zero-interaction items need content.** §7.2’s division stands: at $n_{\text{item}} = 0$ only a content encoder (audio, metadata [69]) places an item; the graph’s cold-start advantage begins at the first event.
- **Directed-case guarantees.** Theorem 4.3 and the sweep-cut guarantee are proven for reversible chains; on directed listening graphs they are inherited procedures with exact invariants (Lemma 4.1) but heuristic entrywise bounds (Remark 4.4). Sharpening this is open.
- **The kernel is only as good as the events.** Direct evaluation has no mechanism for repairing a biased or gamed event stream; the factorized reader’s smoothing sometimes does exactly that.
- **Hybrids are the frontier, not a compromise.** A content encoder can *propose* edges — a k NN graph over audio embeddings, Proposition 3.6 — which the graph store then ingests as TTL’d contributions, decays, and traverses with certified local readers; the RecSys Challenge winner [63] is precisely a factorized-recall / neighborhood-precision hybrid. Nothing in §2 distinguishes an asserted edge from a proposed one but its TTL.

- **Learning the reweighting locally is open.** §8.1 prices the gap between a fixed hub discount and EASE’s globally learned one at roughly $0.323 \rightarrow 0.389$ NDCG@100 on MSD. A reweighting with EASE-like accuracy that admits $O(1)$ local updates — keeping C1–C3 — would close the last accuracy argument for the factorized pipeline on interaction data.

11. Conclusion

“Graph-based” and “embedding-based” music recommendation are not rival theories of musical similarity; they are rival *evaluation strategies* for one mathematical object, the walk kernel $M = \sum_k c_k P^k$ over the interaction graph. The embedding pipeline factorizes a transform of M once into $\mathbb{R}^{n \times d}$ and reads inner products through an ANN index that is itself a graph traversal; the direct pipeline never factorizes and reads $M(t)$ locally at query time, with work independent of catalog size. The correspondence is exact on near-low-rank symmetric kernels — and the hub-suppression corrections that make either reader accurate are the same operation (§5) — while the divergences are the rank and asymmetry obstructions, the completion-versus-fidelity trade, and, decisively for operations, the factorization stage’s global cost, sample threshold, and frozen clock. For event-sourced, decaying, relation-defined interaction data — the data music platforms actually have — the direct reader dominates on complexity, cold start, and non-stationarity, and the published record of the recommender-systems community is consistent with all three claims. For stationary content similarity and zero-interaction items, the factorized reader is irreplaceable. The two compose, and the strongest known systems already are compositions.

References

- [1] L. Katz. *A new status index derived from sociometric analysis*. Psychometrika 18(1), 1953.
- [2] L. Page, S. Brin, R. Motwani, T. Winograd. *The PageRank citation ranking: Bringing order to the Web*. Stanford InfoLab TR, 1999.
- [3] T. Haveliwala. *Topic-sensitive PageRank*. WWW 2002.
- [4] G. Jeh, J. Widom. *Scaling personalized web search*. WWW 2003.
- [5] R. Andersen, F. Chung, K. Lang. *Local graph partitioning using PageRank vectors*. FOCS 2006.
- [6] P. Berkhin. *Bookmark-coloring algorithm for personalized PageRank computing*. Internet Mathematics 3(1), 2006.
- [7] L. Lovász, M. Simonovits. *The mixing rate of Markov chains, an isoperimetric inequality, and computing the volume*. FOCS 1990.
- [8] R. I. Kondor, J. Lafferty. *Diffusion kernels on graphs and other discrete structures*. ICML 2002.
- [9] F. Fouss, A. Pirotte, J.-M. Renders, M. Saerens. *Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation*. IEEE TKDE 19(3), 2007.
- [10] D. Liben-Nowell, J. Kleinberg. *The link-prediction problem for social networks*. JASIST 58(7), 2007.
- [11] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. *Distributed representations of words and phrases and their compositionality*. NeurIPS 2013.
- [12] O. Levy, Y. Goldberg. *Neural word embedding as implicit matrix factorization*. NeurIPS 2014.
- [13] J. Pennington, R. Socher, C. Manning. *GloVe: Global vectors for word representation*. EMNLP 2014.
- [14] S. Arora, Y. Li, Y. Liang, T. Ma, A. Risteski. *A latent variable model approach to PMI-based word embeddings*. TACL 4, 2016.
- [15] K. W. Church, P. Hanks. *Word association norms, mutual information, and lexicography*. Computational Linguistics 16(1), 1990.

- [16] K. Spärck Jones. *A statistical interpretation of term specificity and its application in retrieval*. Journal of Documentation 28(1), 1972.
- [17] S. Robertson, H. Zaragoza. *The probabilistic relevance framework: BM25 and beyond*. Foundations and Trends in IR 3(4), 2009.
- [18] B. Perozzi, R. Al-Rfou, S. Skiena. *DeepWalk: Online learning of social representations*. KDD 2014.
- [19] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, Q. Mei. *LINE: Large-scale information network embedding*. WWW 2015.
- [20] A. Grover, J. Leskovec. *node2vec: Scalable feature learning for networks*. KDD 2016.
- [21] M. Ou, P. Cui, J. Pei, Z. Zhang, W. Zhu. *Asymmetric transitivity preserving graph embedding (HOPE)*. KDD 2016.
- [22] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, J. Tang. *Network embedding as matrix factorization: Unifying DeepWalk, LINE, PTE, and node2vec*. WSDM 2018. [arXiv:1710.02971](https://arxiv.org/abs/1710.02971)
- [23] A. Tsitsulin, D. Mottin, P. Karras, E. Müller. *VERSE: Versatile graph embeddings from similarity measures*. WWW 2018.
- [24] Ş. Postăvaru, A. Tsitsulin, F. M. G. de Almeida, Y. Tian, S. Lattanzi, B. Perozzi. *InstantEmbedding: Efficient local node representations*. 2020. [arXiv:2010.06992](https://arxiv.org/abs/2010.06992)
- [25] M. Belkin, P. Niyogi. *Laplacian eigenmaps for dimensionality reduction and data representation*. Neural Computation 15(6), 2003.
- [26] Y. Malkov, D. Yashunin. *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*. IEEE TPAMI 42(4), 2020. [arXiv:1603.09320](https://arxiv.org/abs/1603.09320)
- [27] Y. Malkov, A. Ponomarenko, A. Logvinov, V. Krylov. *Approximate nearest neighbor algorithm based on navigable small world graphs*. Information Systems 45, 2014.
- [28] L. Prokhorenkova, A. Shekhovtsov. *Graph-based nearest neighbor search: From practice to theory*. ICML 2020.
- [29] E. Candès, B. Recht. *Exact matrix completion via convex optimization*. Foundations of Computational Mathematics 9, 2009.
- [30] E. Candès, T. Tao. *The power of convex relaxation: Near-optimal matrix completion*. IEEE Trans. Information Theory 56(5), 2010.
- [31] B. Recht. *A simpler approach to matrix completion*. JMLR 12, 2011.
- [32] C. Seshadhri, A. Sharma, A. Stolman, A. Goel. *The impossibility of low-rank representations for triangle-rich complex networks*. PNAS 117(11), 2020. [doi:10.1073/pnas.1911030117](https://doi.org/10.1073/pnas.1911030117)
- [33] S. Chanpuriya, C. Musco, K. Sotiropoulos, C. Tsourakakis. *Node embeddings and exact low-rank representations of complex networks*. NeurIPS 2020.
- [34] A. Tversky. *Features of similarity*. Psychological Review 84(4), 1977.
- [35] L. Vilnis, A. McCallum. *Word representations via Gaussian embedding*. ICLR 2015.
- [36] M. Nickel, D. Kiela. *Poincaré embeddings for learning hierarchical representations*. NeurIPS 2017.
- [37] W. L. Hamilton, J. Leskovec, D. Jurafsky. *Diachronic word embeddings reveal statistical laws of semantic change*. ACL 2016.
- [38] A. Singh, S. J. Subramanya, R. Krishnaswamy, H. V. Simhadri. *FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search*. 2021. [arXiv:2105.09613](https://arxiv.org/abs/2105.09613)
- [39] Y. Cao, J. Yang. *Towards making systems forget with machine unlearning*. IEEE S&P 2015.

- [40] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot. *Machine unlearning*. IEEE S&P 2021.
- [41] E. Cohen, M. Strauss. *Maintaining time-decaying stream aggregates*. PODS 2003.
- [42] A. Schein, A. Popescul, L. Ungar, D. Pennock. *Methods and metrics for cold-start recommendations*. SIGIR 2002.
- [43] B. Sarwar, G. Karypis, J. Konstan, J. Riedl. *Item-based collaborative filtering recommendation algorithms*. WWW 2001.
- [44] G. Linden, B. Smith, J. York. *Amazon.com recommendations: Item-to-item collaborative filtering*. IEEE Internet Computing 7(1), 2003.
- [45] M. Deshpande, G. Karypis. *Item-based top-N recommendation algorithms*. ACM TOIS 22(1), 2004.
- [46] X. Ning, G. Karypis. *SLIM: Sparse linear methods for top-N recommender systems*. ICDM 2011.
- [47] H. Steck. *Embarrassingly shallow autoencoders for sparse data*. WWW 2019. [arXiv:1905.03375](https://arxiv.org/abs/1905.03375)
- [48] M. Gori, A. Pucci. *ItemRank: A random-walk based scoring algorithm for recommender engines*. IJCAI 2007.
- [49] C. Cooper, S. H. Lee, T. Radzik, Y. Siantos. *Random walks in recommender systems: Exact computation and simulations*. WWW 2014 Companion.
- [50] F. Christoffel, B. Paudel, C. Newell, A. Bernstein. *Blockbusters and wallflowers: Accurate, diverse, and scalable recommendations with random walks*. RecSys 2015. [doi:10.1145/2792838.2800180](https://doi.org/10.1145/2792838.2800180)
- [51] B. Paudel, F. Christoffel, C. Newell, A. Bernstein. *Updatable, accurate, diverse, and scalable recommendations for interactive applications*. ACM TiS 7(1), 2017. [doi:10.1145/2955101](https://doi.org/10.1145/2955101)
- [52] C. Eksombatchai, P. Jindal, J. Z. Liu, Y. Liu, R. Sharma, C. Sugnet, M. Ulrich, J. Leskovec. *Pixie: A system for recommending 3+ billion items to 200+ million users in real-time*. WWW 2018. [arXiv:1711.07601](https://arxiv.org/abs/1711.07601)
- [53] O. Barkan, N. Koenigstein. *Item2vec: Neural item embedding for collaborative filtering*. IEEE MLSP 2016.
- [54] M. Grbovic, V. Radosavljevic, N. Djuric, N. Bhamidipati, J. Savla, V. Bhagwan, D. Sharp. *E-commerce in your inbox: Product recommendations at scale*. KDD 2015.
- [55] B. Hidasi, A. Karatzoglou, L. Baltrunas, D. Tikk. *Session-based recommendations with recurrent neural networks*. ICLR 2016.
- [56] D. Jannach, M. Ludewig. *When recurrent neural networks meet the neighborhood for session-based recommendation*. RecSys 2017.
- [57] M. Ludewig, D. Jannach. *Evaluation of session-based recommendation algorithms*. User Modeling and User-Adapted Interaction 28(4–5), 2018. [arXiv:1803.09587](https://arxiv.org/abs/1803.09587)
- [58] M. Ludewig, I. Kamehkhosh, N. Landia, D. Jannach. *Effective nearest-neighbor music recommendations*. Proc. ACM RecSys Challenge 2018.
- [59] M. Quadrona, P. Cremonesi, D. Jannach. *Sequence-aware recommender systems*. ACM Computing Surveys 51(4), 2018.
- [60] G. Bonnin, D. Jannach. *Automated generation of music playlists: Survey and experiments*. ACM Computing Surveys 47(2), 2014.
- [61] C.-W. Chen, P. Lamere, M. Schedl, H. Zamani. *RecSys Challenge 2018: Automatic music playlist continuation*. RecSys 2018.

- [62] H. Zamani, M. Schedl, P. Lamere, C.-W. Chen. *An analysis of approaches taken in the ACM RecSys Challenge 2018 for automatic music playlist continuation*. ACM TIST 10(5), 2019. [arXiv:1810.01520](#)
- [63] M. Volkovs, H. Rai, Z. Cheng, G. Wu, Y. Lu, S. Sanner. *Two-stage model for automatic playlist continuation at scale*. Proc. ACM RecSys Challenge 2018. [doi:10.1145/3267471.3267480](#)
- [64] M. Ferrari Dacrema, P. Cremonesi, D. Jannach. *Are we really making much progress? A worrying analysis of recent neural recommendation approaches*. RecSys 2019. [arXiv:1907.06902](#)
- [65] S. Rendle, W. Krichene, L. Zhang, J. Anderson. *Neural collaborative filtering vs. matrix factorization revisited*. RecSys 2020.
- [66] Y. Koren. *Collaborative filtering with temporal dynamics*. KDD 2009.
- [67] Y. Ding, X. Li. *Time weight collaborative filtering*. CIKM 2005.
- [68] J. Vinagre, A. M. Jorge, J. Gama. *Fast incremental matrix factorization for recommendation with positive-only feedback*. UMAP 2014.
- [69] A. van den Oord, S. Dieleman, B. Schrauwen. *Deep content-based music recommendation*. NeurIPS 2013. [papers.nips.cc/paper/5004](#)
- [70] Ò. Celma. *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space*. Springer, 2010.
- [71] M. Schedl, H. Zamani, C.-W. Chen, Y. Deldjoo, M. Elahi. *Current challenges and visions in music recommender systems research*. International Journal of Multimedia Information Retrieval 7(2), 2018.
- [72] D. Fogaras, B. Rácz, K. Csalogány, T. Sarlós. *Towards scaling fully personalized PageRank: Algorithms, lower bounds, and experiments*. Internet Mathematics 2(3), 2005.
- [73] C. Eckart, G. Young. *The approximation of one matrix by another of lower rank*. Psychometrika 1(3), 1936.
- [74] Y. Koren, R. Bell, C. Volinsky. *Matrix factorization techniques for recommender systems*. IEEE Computer 42(8), 2009.
- [75] Y. Hu, Y. Koren, C. Volinsky. *Collaborative filtering for implicit feedback datasets*. ICDM 2008.
- [76] S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme. *BPR: Bayesian personalized ranking from implicit feedback*. UAI 2009.
- [77] D. Liang, R. G. Krishnan, M. D. Hoffman, T. Jebara. *Variational autoencoders for collaborative filtering*. WWW 2018. [arXiv:1802.05814](#)